DIABETES RISK PREDICTION

Professional Data Analytics Project Documentation

Focus: Diabetes Risk Prediction

Academic Data Analytics Project

Prepared by: Sagnik Chowdhury

Stream : BCA

Sec : 1

Roll No : 25342724094.

Supreme Knowledge Foundation Group of Institutions.

Pure Python Script-Based | Modular | Configuration-Driven | Reusable

DOCUMENT PROFILE

Item	Details
Project Focus	Diabetes Risk Prediction
Project Type	Academic Data Analytics Project
Implementation Rule	Pure Python scripts (.py files only)
Primary Model	Logistic Regression
Dataset	Pima Indians Diabetes Dataset
Input	diabetes.csv
Core Stages	Data loading → cleaning → EDA → visualization → modelling → evaluation → risk calculator
Reference Basis	Supplied 20-page Chronic Disease Risk Prediction PDF
Prepared By	Sagnik Chowdhury
Institution	Supreme Knowledge Foundation Group of Institutions.

Documentation objective

This version expands the supplied reference into a structured professional documentation format and incorporates the mandatory coding and submission standards. Charts are included as presentation examples; where numerical values are not provided by the reference, they are explicitly labelled illustrative and must be replaced by values from the actual local project run.

ABSTRACT

Chronic diseases such as diabetes require early identification of relevant risk factors and preventive attention. The supplied reference presents a healthcare analytics workflow that explores patient data, cleans invalid measurements, performs exploratory analysis, builds a classification model and provides a simple risk calculator.

This project documentation follows that workflow while emphasizing professional Python engineering. The implementation is intentionally designed without Jupyter Notebook, Google Colab or any notebook-based environment. The system uses pure Python scripts, modules, functions, centralized configuration, reusable utilities and a clean separation between data loading, preprocessing, analysis, visualization and model execution.

The project uses the Pima Indians Diabetes Dataset with eight input variables and an Outcome target. Logistic Regression is used as the baseline classifier. Model evaluation includes accuracy, precision, recall, F1-score and a confusion matrix. The risk calculator returns a model-estimated probability and a project-interface risk category.

The output is educational and must not be interpreted as a medical diagnosis. This boundary is retained from the supplied reference.

TABLE OF CONTENTS

1. Introduction and Project Context
2. Problem Statement and Objectives
3. Scope and Expected Outcomes
4. Dataset Description
5. Data Understanding and Quality Checks
6. Data Cleaning and Preprocessing
7. Exploratory Data Analysis
8. Bar Chart Analysis
9. Pie Chart Analysis
10. Line Chart Analysis
11. Correlation and Feature Analysis
12. Predictive Modelling – Logistic Regression
13. Model Evaluation
14. Diabetes Risk Calculator
15. System Architecture
16. Mandatory Coding Standards
17. Project Folder Structure
18. Module Responsibilities and Reusability
19. Execution Workflow and Deliverables
20. Dashboard, Limitations and Future Enhancements
21. Detailed Findings
22. Detailed Conclusion
23. Final Submission Checklist

1. INTRODUCTION AND PROJECT CONTEXT

The supplied reference describes a Chronic Disease Risk Prediction and Healthcare Analytics Dashboard focused on diabetes risk prediction. It frames the project as an educational analytics system combining data collection, cleaning, exploratory analysis, visualization, predictive modelling and a simple risk calculator.

Professional project orientation

The requested submission is expected to follow industry-based coding and delivery standards. Therefore, the documentation treats the project as a small software system rather than a one-off analysis. The analytical workflow is preserved, but responsibilities are separated into modules.

Project value

- Convert raw patient records into structured analytical information.
- Investigate candidate diabetes-related risk factors.
- Generate repeatable charts and analytical summaries.
- Build a transparent binary classification baseline.
- Provide a simple model-estimated risk interface.
- Create a foundation that can later support dashboards and additional models.

Reference-aligned workflow

The source workflow can be summarized as: understand the dataset → clean invalid or missing measurements → analyze risk factors → train a classification model → evaluate it → present results through an understandable interface.

2. PROBLEM STATEMENT AND OBJECTIVES

Problem statement

Healthcare teams may have large numbers of patient records. A data analytics system can help identify patterns associated with diabetes and highlight records that may require preventive attention. The technical challenge is to make the workflow reproducible, interpretable and maintainable.

Primary objectives

24. Understand the public diabetes dataset.
25. Clean missing or unrealistic measurements.
26. Study glucose, BMI, age and blood pressure in relation to outcome.
27. Train a Logistic Regression classification model.
28. Evaluate the model using accuracy, precision, recall and F1-score.
29. Create a confusion matrix for model-error analysis.
30. Build a simple diabetes risk calculator.
31. Generate visual outputs programmatically from Python scripts.

Software objectives

- Use configuration instead of hidden hard-coded values.
- Apply the Single Responsibility Principle.
- Keep business logic separate from data access and visualization.
- Handle common failures gracefully.
- Make the code readable, maintainable and reusable.

3. SCOPE AND EXPECTED OUTCOMES

In scope

- CSV ingestion and schema validation.
- Missing/invalid-value handling.
- Exploratory data analysis.
- Bar, pie and line chart generation.
- Correlation analysis.
- Logistic Regression modelling.
- Classification metrics and confusion matrix.
- Educational risk calculator.
- Output management and documentation.

Out of scope

- Clinical diagnosis.
- Treatment recommendation.
- Unvalidated clinical deployment.
- Claiming fixed model accuracy without local execution.
- Treating project-interface probability bands as medical thresholds.

Expected outcome

A runnable Python project should execute through main.py, read the configured dataset, perform preprocessing, generate analysis and charts, train/evaluate the model and provide the defined risk-calculator workflow.

4. DATASET DESCRIPTION

The reference uses the Pima Indians Diabetes Dataset as a commonly used educational binary-classification dataset. It contains eight input variables and an Outcome target.

Variable	Role	Type
Pregnancies	Feature	Numeric
Glucose	Feature	Numeric
BloodPressure	Feature	Numeric
SkinThickness	Feature	Numeric
Insulin	Feature	Numeric
BMI	Feature	Numeric
DiabetesPedigreeFunction	Feature	Numeric
Age	Feature	Numeric
Outcome	Target	Binary: 0 / 1

Dataset file

The runnable project expects a CSV named diabetes.csv. The source also describes a downloader helper for obtaining the CSV when internet access is available. For a professional submission, the local data path should be managed through configuration rather than hard-coded inside analysis functions.

Data responsibility

The dataset is an input artifact. Source code should not contain a manually copied dataset or fixed analytical results.

5. DATA UNDERSTANDING AND QUALITY CHECKS

Inspection sequence

32. Load CSV.
33. Confirm row and column counts.
34. Verify required column names.
35. Inspect data types.
36. Calculate missing-value counts.
37. Calculate duplicate records.
38. Review descriptive statistics.
39. Inspect distributions and suspicious measurements.
40. Prepare features and target.

Quality gates

Quality Area	Check	Response
Schema	Required columns present	Clear validation error if not
Types	Numeric fields parse	Report conversion problems
Missingness	Null/invalid values	Documented imputation strategy
Duplicates	Repeated records	Review and handle consistently
Target	Binary Outcome	Reject unexpected values

Reproducibility

The source emphasizes that later charts and models depend on input quality. A reproducible workflow therefore validates data before analysis and keeps preprocessing decisions documented.

6. DATA CLEANING AND PREPROCESSING

The reference notes that some diabetes datasets contain zero values in fields where zero may not be physiologically meaningful, including glucose, blood pressure, BMI, insulin and skin thickness. The project therefore treats selected zeros as missing and uses median imputation in the modelling pipeline.

Cleaning steps

41. Detect missing values.
42. Detect duplicates.
43. Review selected zero measurements.
44. Convert relevant fields to numeric types.
45. Separate features and target.
46. Split training and testing data.
47. Fit median imputation using training data.
48. Standardize numeric features.
49. Train the classifier through the prepared pipeline.

Why a pipeline?

Keeping imputation and scaling inside the model pipeline reduces the risk of fitting preprocessing transformations using the test set. It also ensures that new risk-calculator inputs receive the same transformations as training data.

Processing flow

Raw CSV → Validation → Cleaning → Train/Test Split → Median Imputation → Scaling → Logistic Regression → Prediction → Evaluation

7. EXPLORATORY DATA ANALYSIS

EDA is the central descriptive analytics stage. It helps investigate the distribution of outcomes and the behaviour of glucose, BMI, age and blood pressure.

Core questions

- How common is each Outcome class?
- Does glucose distribution differ between Outcome groups?
- Does observed outcome rate vary by BMI category?
- Does observed outcome rate vary by age group?
- How do blood-pressure measurements compare between groups?
- Which variables show stronger linear associations?

Charting principle

Charts should be generated by visualization.py and saved into outputs/. The report should distinguish between charts generated from the student's actual dataset and illustrative charts used only to demonstrate documentation design.

8. BAR CHART ANALYSIS

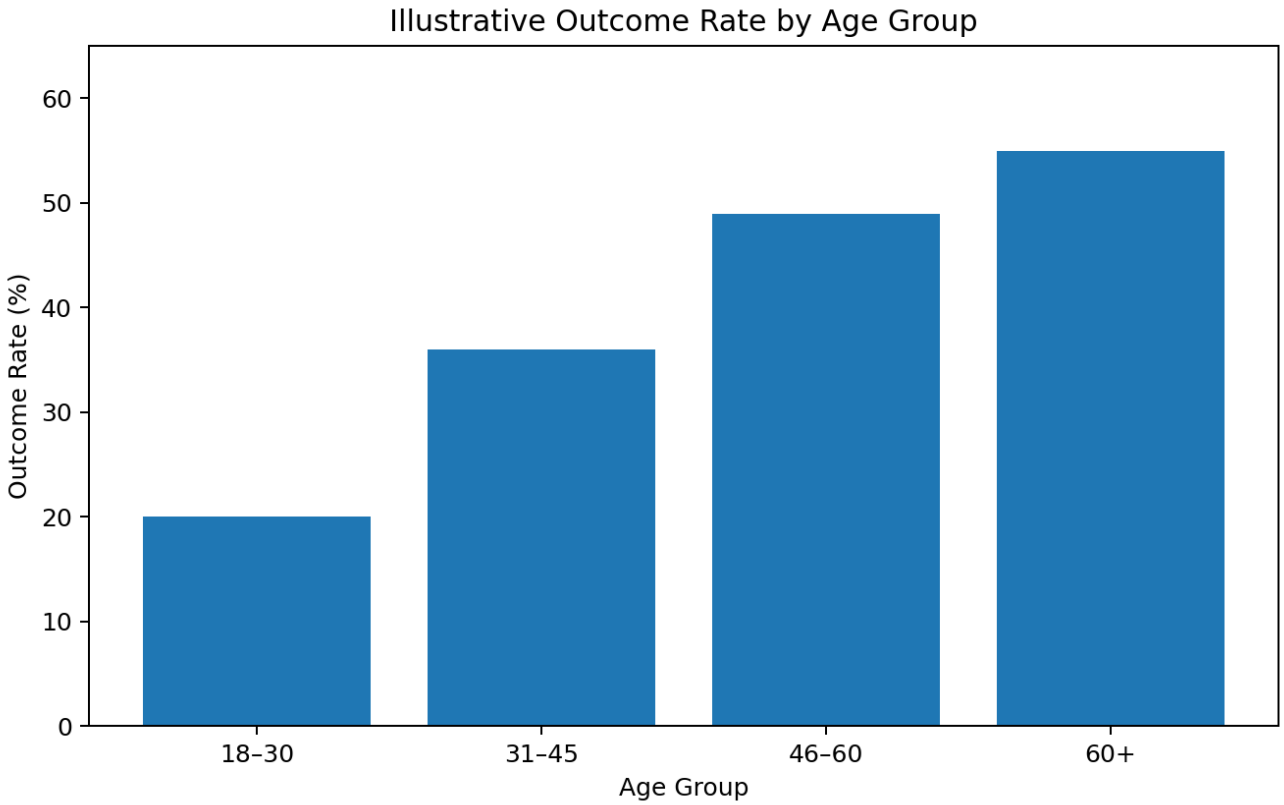


Figure 8.1 — Illustrative age-group outcome rates. Values are presentation examples, not reported results from the supplied PDF.

The supplied reference recommends age-group analysis using ranges such as 18–30, 31–45, 46–60 and 60+. A bar chart is suitable for comparing the observed outcome rate across these categories.

Interpretation method

When the final project is executed, the values in this chart should be calculated from the actual dataset. The analyst should report the observed pattern without presenting it as a clinical threshold or causal relationship.

9. GLUCOSE BAR CHART

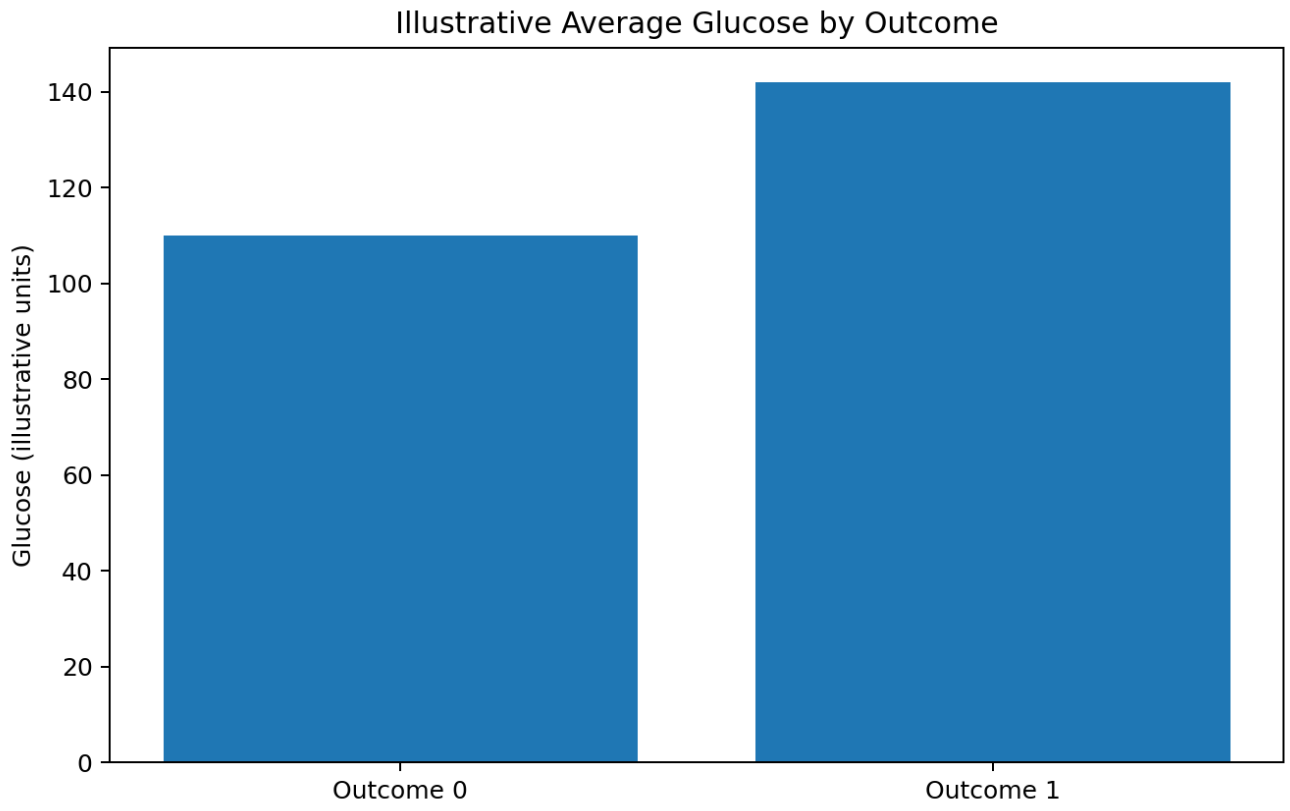


Figure 9.1 — Illustrative comparison of average glucose by outcome. Replace with locally calculated values for final submission.

The reference identifies glucose as an important variable to investigate and recommends comparing measurements with diabetes outcome. A grouped or outcome-level bar chart can make differences in central tendency easy to communicate.

Analytical caution

A visible difference between groups is an observed dataset relationship. It does not by itself prove that glucose is the cause of the outcome.

10. PIE CHART – OUTCOME DISTRIBUTION

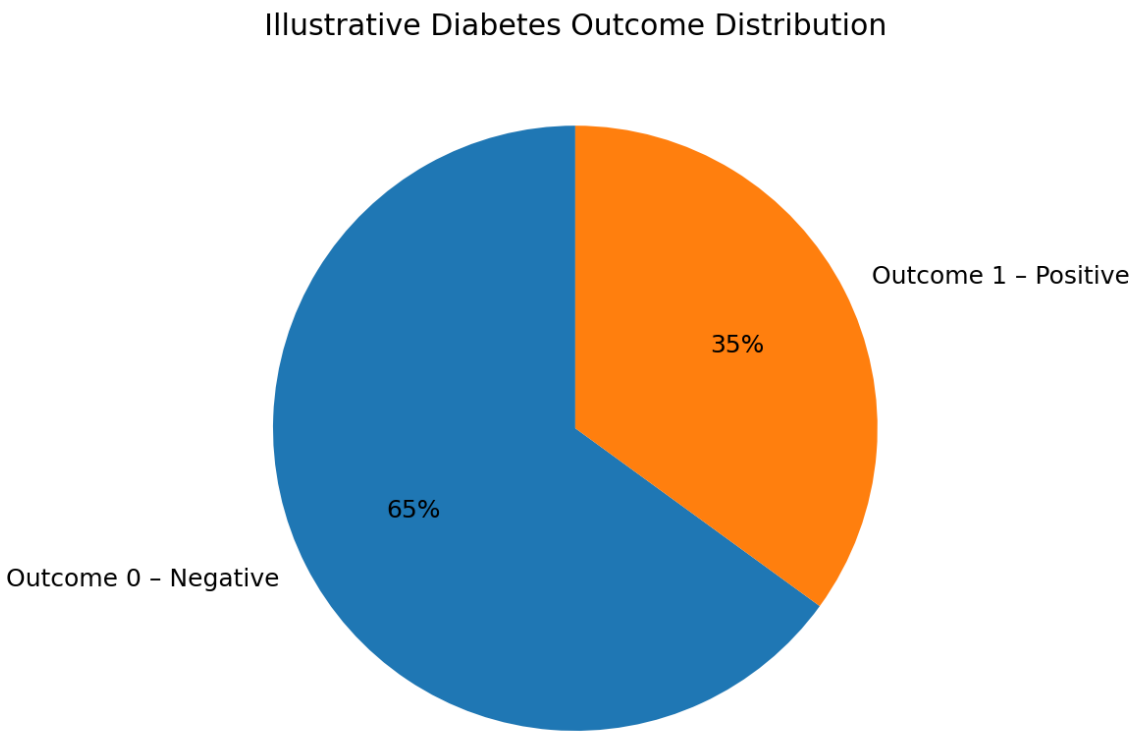


Figure 10.1 — Illustrative outcome distribution. The percentages are placeholders for demonstrating the required pie-chart format.

The target variable is binary: 0 represents the negative class and 1 represents the positive class. The reference recommends a bar chart for class balance; this documentation additionally includes a pie chart because the requested Word report requires one.

Production reporting rule

The final submitted pie chart should be generated from the actual diabetes.csv used for the project. The illustrative percentages shown here should not be presented as the student's measured dataset result.

11. LINE CHART ANALYSIS

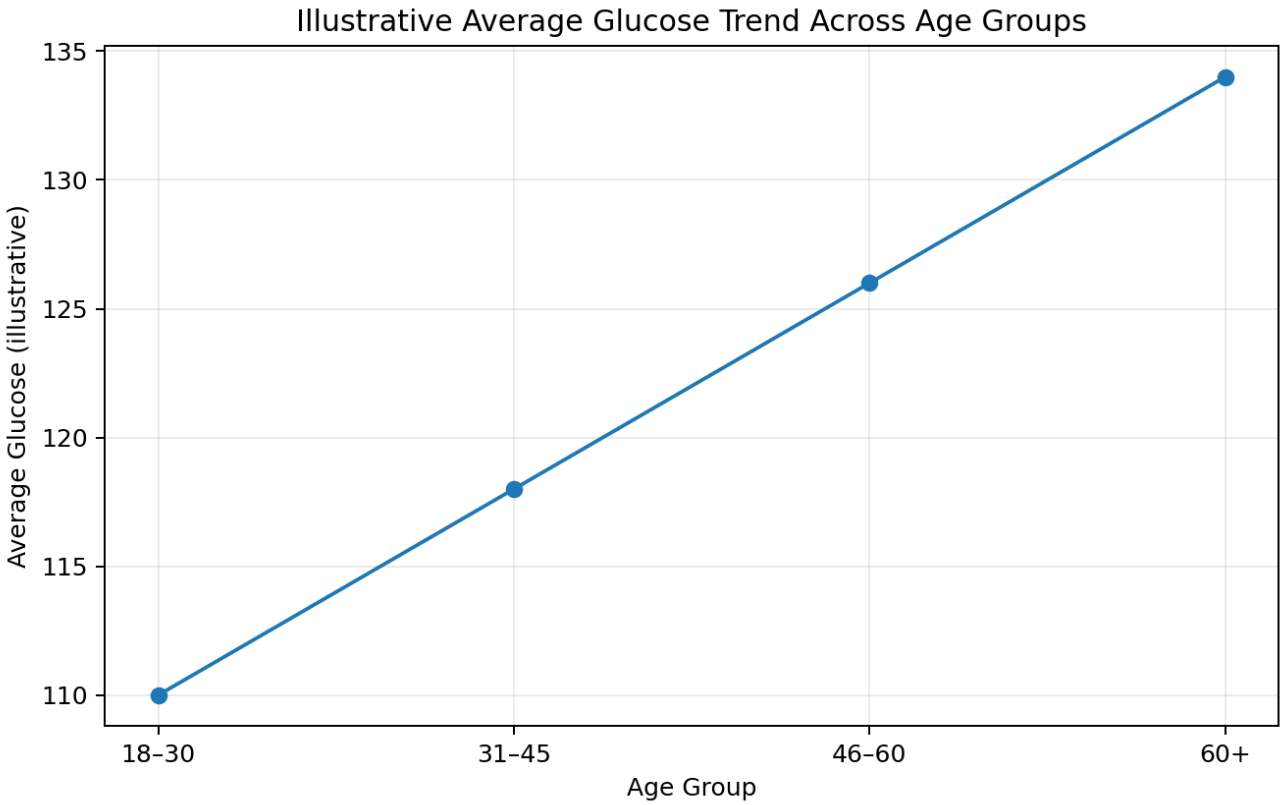


Figure 11.1 — Illustrative average-glucose trend across age groups. Values are demonstration values only.

The source recommends age-group analysis but does not provide numerical age-group glucose averages. A line chart can therefore be used in the final project to show an ordered trend only after the values have been calculated from the local dataset.

Why include a line chart?

- It communicates movement across ordered age groups.
- It can complement the age-group bar chart.
- It provides another visual form for communicating analytical patterns.
- It should not be interpreted as a time-series trend because age groups are categories, not dates.

12. CORRELATION AND FEATURE ANALYSIS

Correlation matrix

A correlation matrix summarizes linear relationships among numeric variables. A heatmap can make stronger and weaker relationships easier to compare.

Interpretation

- Correlation identifies association, not causation.
- The target should be interpreted carefully.
- Feature relationships should be considered alongside distributions and model performance.
- Feature importance can be discussed using standardized Logistic Regression coefficients when appropriate.

Transition to modelling

Correlation and feature analysis provide a bridge from descriptive analytics to predictive modelling. The model should still be evaluated empirically rather than selected solely because a variable appears strongly associated.

13. PREDICTIVE MODELLING – LOGISTIC REGRESSION

The supplied project uses Logistic Regression as a clear baseline classification model. The dataset is split into training and testing sets using stratification, followed by median imputation, feature standardization and classification.

Workflow

Dataset → Stratified Train/Test Split → Median Imputation → Standard Scaling → Logistic Regression → Prediction → Evaluation

Why Logistic Regression?

- Suitable for binary classification.
- Produces an estimated positive-class probability.
- Relatively straightforward to explain.
- Provides a useful baseline before comparing more complex models.

No copied performance

The source explicitly states that a fixed accuracy value should not be copied into the final submission. The final report should record the values generated by the student's own environment.

14. MODEL EVALUATION

Metric	Meaning
Accuracy	Overall correctness
Precision	Correctness among predicted positives
Recall	Detection of actual positive cases
F1-score	Balance between precision and recall
Confusion Matrix	Counts of true/false predictions

Local result entry

Accuracy: _____

Precision: _____

Recall: _____

F1-score: _____

Test-set size: _____

Interpretation

Evaluation metrics describe model behaviour on the selected test set. They do not establish clinical validity. Model performance can change with preprocessing choices, dataset composition and train/test split.

15. DIABETES RISK CALCULATOR

The calculator accepts the eight model features: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction and Age.

Input	Example from reference
Age	52
BMI	32
Glucose	155
Blood Pressure	145
Pregnancies	2

Output

The calculator returns a model-estimated risk category and estimated probability.

Probability	Project-interface category
< 40%	LOW
40%–69.9%	MEDIUM
70% or above	HIGH

These risk bands are project-interface categories, not medical diagnostic thresholds.

Input validation

- Reject non-numeric values where numeric input is required.
- Handle missing fields clearly.
- Apply the same preprocessing used during model training.
- Return a clear validation message instead of an unhandled traceback.

16. SYSTEM ARCHITECTURE

Module	Primary Responsibility
main.py	Entry point and workflow orchestration
config.py	Configuration and constants
data_loader.py	CSV loading and validation
preprocessing.py	Cleaning and model-ready transformations
analysis.py	EDA calculations and metrics
visualization.py	Charts and visual outputs
utils.py	Reusable helpers, validation and common support

Separation of concerns

- Data loading does not create charts.
- Visualization does not perform raw-data ingestion.
- Model training does not own user-interface presentation.
- Configuration stores changeable values.
- main.py coordinates the modules instead of becoming a monolithic implementation.

17. MANDATORY CODING STANDARDS & COMPLIANCE

Strictly prohibited

- Jupyter Notebook (.ipynb)
- Google Colab
- Any notebook-based environment
- Hard-coded values without configuration
- Monolithic single-file scripts without structure

Mandatory requirements

- Pure Python scripts (.py files only)
- Modular programming principles
- Functions and modules
- Clean project structure
- Readable, maintainable and reusable code
- Standard Python coding conventions

Coding standards

- Meaningful function and variable names
- Single Responsibility Principle
- Proper docstrings and comments
- Graceful error handling
- Business logic separate from data loading and visualization
- Reusable and easy-to-maintain code

18. PROJECT FOLDER STRUCTURE

```
project_name/
│
├── main.py              # Entry point
├── config.py            # Configuration & constants
├── data/
│   ├── raw/
│   └── processed/
├── src/
│   ├── data_loader.py
│   ├── preprocessing.py
│   ├── analysis.py
│   ├── visualization.py
│   └── utils.py
├── requirements.txt
├── README.md
└── outputs/
```

Module design

Each module should expose small, testable functions. Paths and constants belong in config.py. Generated files belong in outputs/. Raw and processed data should remain separated.

Reusable design

A clean module boundary allows the same analysis functions to be reused when a different CSV is supplied, subject to schema compatibility and documented preprocessing assumptions.

19. EXECUTION WORKFLOW AND DELIVERABLES

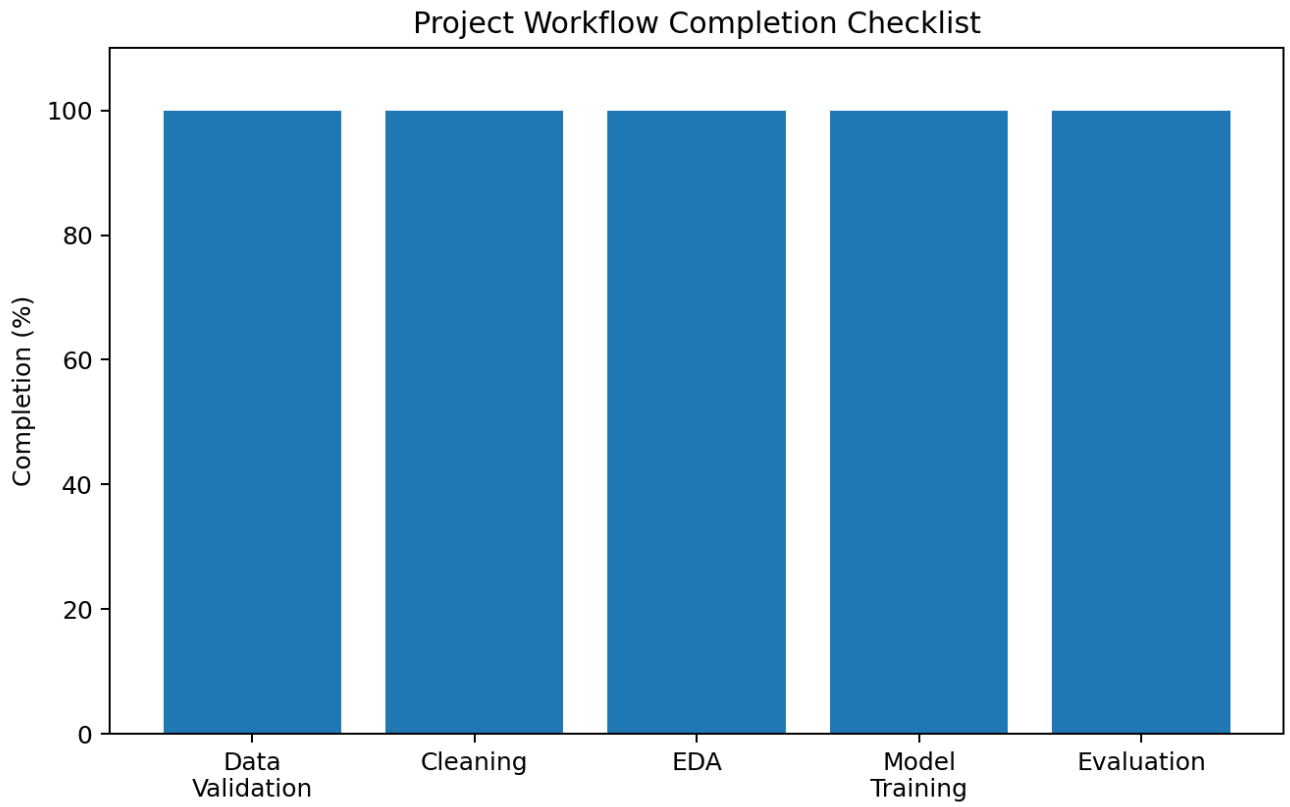


Figure 19.1 — Workflow checklist visualization. This chart represents implementation stages, not model performance.

50. Prepare the project folder and configuration.
51. Place diabetes.csv in the configured raw-data location.
52. Install requirements.txt dependencies.
53. Run main.py.
54. Validate and inspect the data.
55. Apply preprocessing.
56. Generate EDA and charts.
57. Train Logistic Regression.
58. Evaluate model performance.
59. Run the risk calculator.
60. Review outputs/.
61. Record actual locally generated metrics in the final report.

Submission package

- main.py
- config.py
- src/*.py
- data/raw/diabetes.csv
- requirements.txt
- README.md
- outputs/
- Project documentation

20. DASHBOARD, LIMITATIONS AND FUTURE ENHANCEMENTS

Suggested dashboard flow

Patient Data → Data Analytics → Risk Prediction → Risk Factors

Dashboard sections

- Executive Overview: total patients, outcome distribution and average measurements.
- Risk Factor Analysis: age, BMI, glucose and blood pressure visuals.
- High-Risk Analysis: filtered records.
- Model Performance: confusion matrix and evaluation metrics.
- Prediction Form: model-estimated probability.

Limitations

- The dataset is limited and may not represent all populations.
- A model probability is not a medical diagnosis.
- Model performance depends on data and preprocessing.
- Clinical deployment requires validation, governance, privacy controls and expert review.

Future enhancements

62. Add larger and more diverse datasets.
63. Compare Logistic Regression, Decision Tree and Random Forest.
64. Add Streamlit or Power BI dashboards.
65. Add model explainability and feature-importance visuals.
66. Add secure database storage for approved academic demonstrations.
67. Extend the framework to heart disease or hypertension datasets.

```
import pandas as pd
import matplotlib.pyplot as plt



FILE_PATH = "chronic_disease_risk_prediction_detailed.csv"



def load_data(file_path):
    """Load the CSV file into a pandas DataFrame."""
    return pd.read_csv(file_path)



def show_dataset_info(df):
    """Display basic information about the dataset."""
    print("=" * 70)
    print("CHRONIC DISEASE RISK ANALYSIS")
    print("=" * 70)
    print(f"Rows: {df.shape[0]}")
    print(f"Columns: {df.shape[1]}")
    print("\nColumn Names:")
    print(list(df.columns))

    print("\nData Types:")
    print(df.dtypes)

    print("\nMissing Values:")
    print(df.isnull().sum())

    print("\nFirst 5 Records:")
    print(df.head().to_string(index=False))
```

```
def descriptive_analysis(df):
    """Display descriptive statistics for important numeric columns."""
    numeric_columns = [
        "Age", "BMI", "Glucose", "BloodPressure",
        "Cholesterol", "RiskScore"
    ]

    print("\n" + "=" * 70)
    print("DESCRIPTIVE STATISTICS")
    print("=" * 70)
    print(df[numeric_columns].describe().round(2))



def category_analysis(df):
    """Analyze disease, risk, gender and other categorical variables."""
    print("\n" + "=" * 70)
    print("CATEGORICAL ANALYSIS")
    print("=" * 70)

    print("\nDisease Distribution:")
    print(df["Disease"].value_counts())

    print("\nRisk Level Distribution:")
    print(df["RiskLevel"].value_counts())

    print("\nGender Distribution:")
    print(df["Gender"].value_counts())
```

```
    print("\nBMI Category:")
    print(df["BMICategory"].value_counts())

    print("\nGlucose Category:")
    print(df["GlucoseCategory"].value_counts())

    print("\nBlood Pressure Category:")
    print(df["BPCategory"].value_counts())


def grouped_analysis(df):
    """Display grouped statistics."""
    print("\n" + "=" * 70)
    print("GROUPED ANALYSIS")
    print("=" * 70)

    print("\nAverage Risk Score by Disease:")
    result = df.groupby("Disease")["RiskScore"].agg(
        Count="count",
        Average="mean",
        Minimum="min",
        Maximum="max"
    )
    print(result.round(2))

    print("\nDisease vs Risk Level:")
    print(pd.crosstab(df["Disease"], df["RiskLevel"]))


def top_risk_patients(df, n=5):
```

```
    """Display patients with the highest risk scores."""
    print("\n" + "=" * 70)
    print(f"TOP {n} HIGHEST-RISK PATIENTS")
    print("=" * 70)

    columns = [
        "Patient_ID", "Age", "Gender", "Glucose",
        "BMI", "Disease", "RiskScore", "RiskLevel"
    ]

    result = df.nlargest(n, "RiskScore")[columns]
    print(result.to_string(index=False))


def correlation_analysis(df):
    """Display correlations between important numeric variables."""
    columns = [
        "Age", "BMI", "Glucose",
        "BloodPressure", "Cholesterol", "RiskScore"
    ]

    print("\n" + "=" * 70)
    print("CORRELATION MATRIX")
    print("=" * 70)
    print(df[columns].corr().round(2))


def create_charts(df):
    """Create and display analytical charts."""
```

```
# 1. Disease distribution
plt.figure(figsize=(7, 5))
df["Disease"].value_counts().plot(kind="bar")
plt.title("Disease Distribution")
plt.xlabel("Disease")
plt.ylabel("Number of Patients")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# 2. Risk level distribution
plt.figure(figsize=(7, 5))
df["RiskLevel"].value_counts().plot(kind="bar")
plt.title("Risk Level Distribution")
plt.xlabel("Risk Level")
plt.ylabel("Number of Patients")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# 3. Age distribution
plt.figure(figsize=(8, 5))
plt.hist(df["Age"], bins=10)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Number of Patients")
plt.tight_layout()
plt.show()

# 4. Risk score distribution
```

```
plt.figure(figsize=(8, 5))
plt.hist(df["RiskScore"], bins=10)
plt.title("Risk Score Distribution")
plt.xlabel("Risk Score")
plt.ylabel("Number of Patients")
plt.tight_layout()
plt.show()

# 5. Average risk score by disease
average_risk = df.groupby("Disease")["RiskScore"].mean()
plt.figure(figsize=(7, 5))
average_risk.plot(kind="bar")
plt.title("Average Risk Score by Disease")
plt.xlabel("Disease")
plt.ylabel("Average Risk Score")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# 6. Disease pie chart
plt.figure(figsize=(7, 7))
df["Disease"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Disease Percentage")
plt.ylabel("")
plt.tight_layout()
plt.show()
```

```
    # 7. Glucose vs Risk Score
    plt.figure(figsize=(8, 5))
    plt.scatter(df["Glucose"], df["RiskScore"], alpha=0.6)
    plt.title("Glucose vs Risk Score")
    plt.xlabel("Glucose")
    plt.ylabel("Risk Score")
    plt.tight_layout()
    plt.show()

    # 8. Blood Pressure vs Risk Score
    plt.figure(figsize=(8, 5))
    plt.scatter(df["BloodPressure"], df["RiskScore"], alpha=0.6)
    plt.title("Blood Pressure vs Risk Score")
    plt.xlabel("Blood Pressure")
    plt.ylabel("Risk Score")
    plt.tight_layout()
    plt.show()



def main():
    """Run the complete analysis."""
    df = load_data(FILE_PATH)

    show_dataset_info(df)
    descriptive_analysis(df)
    category_analysis(df)
    grouped_analysis(df)
    top_risk_patients(df)
    correlation_analysis(df)
```

```
    create_charts(df)



if __name__ == "__main__":
    main()
```

22. DETAILED CONCLUSION

This Diabetes Risk Prediction project provides a complete end-to-end data analytics workflow built around the diabetes-focused reference supplied for the assignment. It begins with structured patient data and proceeds through data understanding, quality checking, cleaning, exploratory analysis, visualization, predictive modelling, model evaluation and a simple risk-calculator interface.

The most important technical improvement in this documentation is the explicit alignment with the mandatory software-engineering requirements. The implementation is designed as a pure Python script project rather than a notebook. The recommended structure separates main.py, configuration, data access, preprocessing, analysis, visualization and utility functions. This separation reduces complexity, supports reuse and makes the project easier to review and maintain.

The data-preparation stage is treated as a first-class part of the solution. The reference identifies selected zero measurements as potentially invalid for physiological interpretation and recommends median imputation. Keeping this logic inside a documented preprocessing pipeline makes the model workflow more reproducible and reduces the risk of inconsistent transformations.

The visualization component strengthens the analytical story. Bar charts can compare age groups and outcome-related measurements; the pie chart provides an intuitive view of class composition; and the line chart can communicate an ordered analytical pattern when the underlying values are calculated from the real dataset. Because the supplied reference does not provide numerical values for these charts, the examples in this report are explicitly illustrative rather than claimed dataset findings.

The Logistic Regression model provides a transparent baseline for binary classification and can produce an estimated probability for the positive class. Accuracy, precision, recall, F1-score and the confusion matrix provide complementary views of model behaviour. The final project should always report values produced by the student's own local execution rather than copying fixed values from an example.

The risk calculator demonstrates how analytics can be connected to a simple user-facing workflow. Nevertheless, its probability and LOW/MEDIUM/HIGH interface categories must remain within the educational scope of the project. They are not clinical diagnostic thresholds and cannot replace professional medical assessment.

From a software-development perspective, the project is designed to be extensible. A future version can add additional algorithms, model explainability, larger datasets, secure data storage and an interactive dashboard without abandoning the modular architecture. This makes the project not only an analytics exercise but also a demonstration of structured Python engineering.

Final conclusion points

The project establishes a complete and reproducible diabetes-risk analytics workflow.

Pure Python scripts satisfy the prohibition on notebook-based implementation.

Modular functions and modules support maintainability and reuse.

Configuration reduces hidden hard-coded values and makes deployment settings easier to change.

Data validation and preprocessing improve the reliability of downstream analysis.

Bar, pie and line charts improve communication of analytical patterns while keeping illustrative values clearly identified.

Logistic Regression provides an interpretable baseline for binary classification.

Multiple evaluation metrics provide a more complete assessment than accuracy alone.

The risk calculator demonstrates practical use of a model-estimated probability while maintaining the required medical-use limitation.

The architecture provides a foundation for future dashboards, additional models and broader chronic-disease analytics.

Final submission checklist

☐ Only .py implementation files are used.

☐ No Jupyter Notebook / Google Colab dependency.

☐ No monolithic single-file implementation.

☐ Configuration is centralized.

☐ Functions have meaningful names and focused responsibilities.

☐ Errors are handled gracefully.

☐ Data loading, business logic and visualization are separated.

☐ Charts are generated programmatically.

☐ Actual model metrics are recorded from the local run.

☐ README.md and requirements.txt are included.

☐ outputs/ contains generated charts/results.

☐ Documentation contains limitations and the detailed conclusion.